Министерство образования Республики Беларусь
Учреждение образования
«Брестский Государственный технический университет»
Кафедра ИИТ

Лабораторная работа №3
По дисциплине «КиСБ»
Тема: «Блочные шифры и классические режимы шифрования»

Выполнил:
Студент 4 курса
Группы АС-65
Ракецкий П. П.
Проверил:
Хацкевич А. С.

Брест 2026

Цель: программная реализация симметричного блочного шифра (AES-128 или СТБ 34.101.31 «БелТ») и классических режимов работы (ECB, CBC, CFB, OFB, CTR) с поддержкой алгоритма дополнения блоков (Padding).

Вариант 3

Обязательный теоретический минимум:

- архитектура SP-сетей (на примере AES) и модифицированных сетей Фейстеля (на примере БелТ): генерация раундовых ключей, табличные подстановки (S-box), перестановки и смешивание;
- классические режимы шифрования: Electronic Codebook (ECB), Cipher Block Chaining (CBC), Cipher Feedback (CFB), Output Feedback (OFB), Counter (CTR);
- механизмы дополнения неполных блоков по стандарту PKCS#7;
- побитовые и арифметические операции над байтами и 32-битными словами.

Проект должен состоять из двух логически изолированных модулей:

1. **Блочный шифр:** реализация базового алгоритма (AES-128 или БелТ) согласно варианту. Модуль должен инкапсулировать генерацию раундовых ключей и содержать функции зашифрования/расшифрования строго одного 128-битного блока: encrypt_block(pt_block, key), decrypt_block(ct_block, key).
2. **Режимы шифрования:** модуль, использующий атомарные функции из первого модуля для обработки сообщений произвольной длины. Реализация базового режима ECB (обязательно для всех вариантов) и одного дополнительного режима (CBC, CFB, OFB или CTR). Модуль также должен включать функции наложения и снятия паддинга.

Aes_block.py:

```
# AES-128: только атомарные операции над одним 128-битным блоком.
# Без сторонних криптографических библиотек.

SBOX = [

0x63,0x7c,0x77,0x7b,0xf2,0x6b,0x6f,0xc5,0x30,0x01,0x67,0x2b,0xfe,0xd7,0xab,0x
76,

0xca,0x82,0xc9,0x7d,0xfa,0x59,0x47,0xf0,0xad,0xd4,0xa2,0xaf,0x9c,0xa4,0x72,0x
c0,

0xb7,0xfd,0x93,0x26,0x36,0x3f,0xf7,0xcc,0x34,0xa5,0xe5,0xf1,0x71,0xd8,0x31,0x
15,

0x04,0xc7,0x23,0xc3,0x18,0x96,0x05,0x9a,0x07,0x12,0x80,0xe2,0xeb,0x27,0xb2,0x
75,

0x09,0x83,0x2c,0x1a,0x1b,0x6e,0x5a,0xa0,0x52,0x3b,0xd6,0xb3,0x29,0xe3,0x2f,0x
84,

0x53,0xd1,0x00,0xed,0x20,0xfc,0xb1,0x5b,0x6a,0xcb,0xbe,0x39,0x4a,0x4c,0x58,0x
cf,

0xd0,0xef,0xaa,0xfb,0x43,0x4d,0x33,0x85,0x45,0xf9,0x02,0x7f,0x50,0x3c,0x9f,0x
a8,
```

```
0x51,0xa3,0x40,0x8f,0x92,0x9d,0x38,0xf5,0xbc,0xb6,0xda,0x21,0x10,0xff,0xf3,0x
d2,

0xcd,0x0c,0x13,0xec,0x5f,0x97,0x44,0x17,0xc4,0xa7,0x7e,0x3d,0x64,0x5d,0x19,0x
73,

0x60,0x81,0x4f,0xdc,0x22,0x2a,0x90,0x88,0x46,0xee,0xb8,0x14,0xde,0x5e,0x0b,0x
db,

0xe0,0x32,0x3a,0x0a,0x49,0x06,0x24,0x5c,0xc2,0xd3,0xac,0x62,0x91,0x95,0xe4,0x
79,

0xe7,0xc8,0x37,0x6d,0x8d,0xd5,0x4e,0xa9,0x6c,0x56,0xf4,0xea,0x65,0x7a,0xae,0x
08,

0xba,0x78,0x25,0x2e,0x1c,0xa6,0xb4,0xc6,0xe8,0xdd,0x74,0x1f,0x4b,0xbd,0x8b,0x
8a,

0x70,0x3e,0xb5,0x66,0x48,0x03,0xf6,0x0e,0x61,0x35,0x57,0xb9,0x86,0xc1,0x1d,0x
9e,

0xe1,0xf8,0x98,0x11,0x69,0xd9,0x8e,0x94,0x9b,0x1e,0x87,0xe9,0xce,0x55,0x28,0x
df,

0x8c,0xa1,0x89,0x0d,0xbf,0xe6,0x42,0x68,0x41,0x99,0x2d,0x0f,0xb0,0x54,0xbb,0x
16,
]

INV_SBOX = [0] * 256
for i, v in enumerate(SBOX):
    INV_SBOX[v] = i

RCON = [0x01, 0x02, 0x04, 0x08, 0x10, 0x20, 0x40, 0x80, 0x1B, 0x36]


def xtime(b: int) -> int:
    b <<= 1
    if b & 0x100:
        b ^= 0x11B
    return b & 0xFF


def gmul(a: int, b: int) -> int:
    """Умножение в GF(2^8) по модулю x^8+x^4+x^3+x+1."""
    result = 0
    for _ in range(8):
        if b & 1:
            result ^= a
        b >>= 1
        a = xtime(a)
    return result


# ---------- Key Expansion ----------
def key_expansion(key: bytes):
    assert len(key) == 16, "AES-128 требует 16-байтовый ключ"
    w = [list(key[4 * i:4 * i + 4]) for i in range(4)]  # 4 слова по 4 байта
    for i in range(4, 44):
        temp = w[i - 1][:]
        if i % 4 == 0:
            # RotWord
            temp = temp[1:] + temp[:1]
            # SubWord
```

```
            temp = [SBOX[b] for b in temp]
            # Rcon
            temp[0] ^= RCON[i // 4 - 1]
        w.append([w[i - 4][j] ^ temp[j] for j in range(4)])
    # 11 раундовых ключей по 16 байт
    return [bytes(sum(w[4 * r:4 * r + 4], [])) for r in range(11)]


# ---------- Раундовые преобразования ----------
def add_round_key(state, rkey):
    return [state[i] ^ rkey[i] for i in range(16)]


def sub_bytes(state):
    return [SBOX[b] for b in state]


def inv_sub_bytes(state):
    return [INV_SBOX[b] for b in state]


def shift_rows(state):
    # state — плоский список 16 байт, column-major (state[r + 4*c])
    s = state[:]
    for r in range(1, 4):
        row = [s[r + 4 * c] for c in range(4)]
        row = row[r:] + row[:r]  # сдвиг влево на r
        for c in range(4):
            s[r + 4 * c] = row[c]
    return s


def inv_shift_rows(state):
    s = state[:]
    for r in range(1, 4):
        row = [s[r + 4 * c] for c in range(4)]
        row = row[-r:] + row[:-r]  # сдвиг вправо на r
        for c in range(4):
            s[r + 4 * c] = row[c]
    return s


def mix_columns(state):
    s = state[:]
    for c in range(4):
        col = s[4 * c:4 * c + 4]
        s[4 * c + 0] = gmul(col[0], 2) ^ gmul(col[1], 3) ^ col[2] ^ col[3]
        s[4 * c + 1] = col[0] ^ gmul(col[1], 2) ^ gmul(col[2], 3) ^ col[3]
        s[4 * c + 2] = col[0] ^ col[1] ^ gmul(col[2], 2) ^ gmul(col[3], 3)
        s[4 * c + 3] = gmul(col[0], 3) ^ col[1] ^ col[2] ^ gmul(col[3], 2)
    return s


def inv_mix_columns(state):
    s = state[:]
    for c in range(4):
        col = s[4 * c:4 * c + 4]
        s[4 * c + 0] = gmul(col[0], 14) ^ gmul(col[1], 11) ^ gmul(col[2], 13)
^ gmul(col[3], 9)
        s[4 * c + 1] = gmul(col[0], 9)  ^ gmul(col[1], 14) ^ gmul(col[2], 11)
^ gmul(col[3], 13)
        s[4 * c + 2] = gmul(col[0], 13) ^ gmul(col[1], 9)  ^ gmul(col[2], 14)
^ gmul(col[3], 11)
        s[4 * c + 3] = gmul(col[0], 11) ^ gmul(col[1], 13) ^ gmul(col[2], 9)
```

```
^ gmul(col[3], 14)
    return s


# ---------- Публичные атомарные функции ----------
def encrypt_block(pt_block: bytes, key: bytes) -> bytes:
    """Зашифровать ровно один 128-битный блок."""
    assert len(pt_block) == 16, "Блок должен быть 16 байт"
    rkeys = key_expansion(key)
    state = list(pt_block)
    state = add_round_key(state, list(rkeys[0]))
    for r in range(1, 10):
        state = sub_bytes(state)
        state = shift_rows(state)
        state = mix_columns(state)
        state = add_round_key(state, list(rkeys[r]))
    # финальный раунд без MixColumns
    state = sub_bytes(state)
    state = shift_rows(state)
    state = add_round_key(state, list(rkeys[10]))
    return bytes(state)


def decrypt_block(ct_block: bytes, key: bytes) -> bytes:
    """Расшифровать ровно один 128-битный блок (используется только в
ECB)."""
    assert len(ct_block) == 16, "Блок должен быть 16 байт"
    rkeys = key_expansion(key)
    state = list(ct_block)
    state = add_round_key(state, list(rkeys[10]))
    for r in range(9, 0, -1):
        state = inv_shift_rows(state)
        state = inv_sub_bytes(state)
        state = add_round_key(state, list(rkeys[r]))
        state = inv_mix_columns(state)
    state = inv_shift_rows(state)
    state = inv_sub_bytes(state)
    state = add_round_key(state, list(rkeys[0]))
    return bytes(state)
```

Modes.py:

```
# Логика режимов поверх атомарных функций блочного шифра.
from aes_block import encrypt_block, decrypt_block

BLOCK = 16


# ---------- PKCS#7 ----------
def pkcs7_pad(data: bytes, block: int = BLOCK) -> bytes:
    k = block - (len(data) % block)
    return data + bytes([k]) * k


def pkcs7_unpad(data: bytes, block: int = BLOCK) -> bytes:
    if not data or len(data) % block != 0:
        raise ValueError("Некорректная длина дополненного сообщения")
    k = data[-1]
    if not (1 <= k <= block):
        raise ValueError("Некорректное значение паддинга")
    if data[-k:] != bytes([k]) * k:
        raise ValueError("Проверка паддинга не пройдена (Padding Oracle
guard)")
    return data[:-k]
```

```
# ---------- ECB ----------
def ecb_encrypt(pt: bytes, key: bytes) -> bytes:
    pt = pkcs7_pad(pt)
    return b"".join(encrypt_block(pt[i:i + BLOCK], key)
                    for i in range(0, len(pt), BLOCK))


def ecb_decrypt(ct: bytes, key: bytes) -> bytes:
    if len(ct) % BLOCK != 0:
        raise ValueError("Длина шифротекста не кратна блоку")
    pt = b"".join(decrypt_block(ct[i:i + BLOCK], key)
                  for i in range(0, len(ct), BLOCK))
    return pkcs7_unpad(pt)


# ---------- CFB (полноблочный CFB-128) ----------
# Демонстрация того, что и для шифрования, и для расшифрования
# используется ТОЛЬКО функция E_K. Паддинг НЕ нужен — CFB потоковый.
def cfb_encrypt(pt: bytes, key: bytes, iv: bytes) -> bytes:
    assert len(iv) == BLOCK
    out = bytearray()
    prev = iv
    # обрабатываем целыми блоками + последний неполный
    for i in range(0, len(pt), BLOCK):
        chunk = pt[i:i + BLOCK]
        keystream = encrypt_block(prev, key)
        ct_chunk = bytes(a ^ b for a, b in zip(chunk, keystream))
        out += ct_chunk
        prev = ct_chunk if len(ct_chunk) == BLOCK else ct_chunk.ljust(BLOCK,
b"\x00")
    return bytes(out)


def cfb_decrypt(ct: bytes, key: bytes, iv: bytes) -> bytes:
    assert len(iv) == BLOCK
    out = bytearray()
    prev = iv
    for i in range(0, len(ct), BLOCK):
        chunk = ct[i:i + BLOCK]
        keystream = encrypt_block(prev, key)          # только E_K, НЕ D_K!
        pt_chunk = bytes(a ^ b for a, b in zip(chunk, keystream))
        out += pt_chunk
        prev = chunk if len(chunk) == BLOCK else chunk.ljust(BLOCK, b"\x00")
    return bytes(out)
```

Demo.py:

```
import os
from aes_block import encrypt_block, decrypt_block
from modes import (ecb_encrypt, ecb_decrypt, cfb_encrypt, cfb_decrypt,
                   pkcs7_pad, pkcs7_unpad)

KEY = bytes.fromhex("000102030405060708090a0b0c0d0e0f")
IV  = bytes.fromhex("101112131415161718191a1b1c1d1e1f")

# --- Вектор NIST FIPS-197 (AES-128) ---
pt = bytes.fromhex("00112233445566778899aabbccddeeff")
ct = encrypt_block(pt, KEY)
print("AES-128 NIST  ct =", ct.hex())
assert ct.hex() == "69c4e0d86a7b0430d8cdb78070b4c55a"
assert decrypt_block(ct, KEY) == pt
print("AES-128 round-trip OK\n")
```

```
# --- PKCS#7 ---
for L in (0, 5, 16, 17, 32):
    msg = os.urandom(L)
    padded = pkcs7_pad(msg)
    assert len(padded) % 16 == 0
    assert pkcs7_unpad(padded) == msg
    print(f"PKCS#7 len={L:2d} -> padded={len(padded):2d}, last={padded[-
1]:02x}")
print("PKCS#7 OK\n")

# --- ECB ---
for L in (0, 7, 16, 33):
    msg = os.urandom(L)
    c = ecb_encrypt(msg, KEY)
    assert ecb_decrypt(c, KEY) == msg
print("ECB round-trip OK\n")

# --- CFB (без паддинга, длина сохраняется) ---
for L in (0, 1, 7, 16, 17, 33):
    msg = os.urandom(L)
    c = cfb_encrypt(msg, KEY, IV)
    assert len(c) == len(msg), "CFB не должен менять длину"
    assert cfb_decrypt(c, KEY, IV) == msg
print("CFB round-trip OK (padding не нужен)\n")

# --- Демонстрация: ECB vs CFB на повторяющихся паттернах ---
pattern = b"A" * 16 + b"B" * 16 + b"A" * 16 + b"B" * 16
print("ECB:", ecb_encrypt(pattern, KEY).hex())
print("CFB:", cfb_encrypt(pattern, KEY, IV).hex())
# В ECB блоки 'A'*16 повторяются -> одинаковый шифротекст (уязвимость)
# В CFB одинаковые блоки дают разный шифротекст

# --- Демонстрация: только E_K в CFB-расшифровании ---
# Проверим, что cfb_decrypt не вызывает decrypt_block: подменим его
import modes
_orig = modes.decrypt_block
modes.decrypt_block = lambda *a, **k: (_ for _ in ()).throw(RuntimeError("D_K
вызвана!"))
msg = b"CFB decrypt must not call D_K. " * 3
c = cfb_encrypt(msg, KEY, IV)
assert cfb_decrypt(c, KEY, IV) == msg   # исключение не выброшено
modes.decrypt_block = _orig
print("\nCFB-расшифрование использует только E_K — подтверждено")
```

Вывод программы в консоле:

```
AES-128 NIST  ct = 69c4e0d86a7b0430d8cdb78070b4c55a

AES-128 round-trip OK

PKCS#7 len= 0 -> padded=16, last=10

PKCS#7 len= 5 -> padded=16, last=0b

PKCS#7 len=16 -> padded=32, last=10

PKCS#7 len=17 -> padded=32, last=0f

PKCS#7 len=32 -> padded=48, last=10

PKCS#7 OK
```

ECB round-trip OK

CFB round-trip OK (padding не нужен)

ECB:
dd4b1a0b47daa7067d0b59d95d58a6ae87f815ad0e513346e0afe99814b1a504dd4b1a0b47daa7067d0b59d95d58a6ae87f815ad0e513346e0afe99814b1a504954f64f2e4e86e9eee82d20216684899

CFB:
46bfae35a094422fd14faf50cfd5d3d252833a89b2591326b4a889d5041310aba72b2edb8cff330dd2de2baf6286f8e4f042d205664828885f4b54fdefb33700

CFB-расшифрование использует только E_K — подтверждено

Вывод: в ходе выполнения лабораторной работы режим CFB действительно позволяет расшифровывать сообщение, располагая **только** функцией зашифрования блока E_K— это его ключевое свойство и главный фокус данной работы. В отличие от ECB, CFB не требует PKCS#7-паддинга и корректно обрабатывает сообщения произвольной байтовой длины.